

CloudShield AI

11 — AI Safety & Guardrail Specification

Version 1.0 | Product-Level Engineering Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the safety architecture, guardrails, abuse resistance, validation gates, runtime controls, human governance, evaluation requirements, and failure behavior required to use AI safely inside CloudShield AI.

Canonical principle: AI proposes. Deterministic systems validate. Governance decides. Cloud APIs execute. Evidence records. Trusted state is earned.

1. Purpose and Scope

This specification defines the safety controls that constrain probabilistic AI behavior across CloudShield AI. It builds on the Intelligence & AI Architecture and LLM Integration specifications and provides the detailed control layer between model output and consequential product behavior.

- Protect users, tenants, cloud environments, and infrastructure from unsafe AI behavior.
- Prevent prompt injection, indirect prompt injection, data exfiltration, tool abuse, hallucinated changes, and model overreach.
- Define input, context, generation, output, tool, candidate, verification, and promotion guardrails.
- Define human-in-the-loop and policy-driven governance for consequential actions.
- Make AI failures observable, bounded, reversible where possible, and non-authoritative.
- Provide an MVP safety baseline and a path toward stronger agentic and local-model controls.

Out of scope: general application security controls already defined in Specification 08, provider-specific API details already defined in Specification 10, and the deterministic DAIL lifecycle rules defined in the DAIL subsystem specifications. This document defines how AI must interact with those controls.

2. Safety Model

2.1 AI trust classification

AI Output	Trust Level	Allowed Use
Raw model output	Untrusted	Inspection/validation only.
Schema-valid output	Conditionally trusted	Can enter domain validation; not authoritative.
Domain-validated recommendation	Advisory/candidate-generating	Can be shown or converted to an explicit candidate.
Verified candidate	Governed system state	Established only by deterministic verification/DAIL.
Trusted infrastructure state	Authoritative product state	Can only be earned through governed promotion.

No AI-generated content may skip levels in this model.

2.2 Safety properties

- No unsafe model output becomes trusted state.
- No model instruction can override server-side authorization or deterministic policy.
- No model can obtain unrestricted cloud credentials.
- No untrusted content can silently become a system instruction.
- No high-impact action occurs without the required governance and verification gates.
- Safety controls remain active regardless of model provider or model quality.
- Failure of an AI safety control must not silently increase authority.

3. Guardrail Architecture

Guardrail Layer	Primary Function	Failure Behavior
Input Guardrail	Validate user/task input, size, type, and prohibited request patterns	Reject or constrain request.
Context Guardrail	Authorization, minimization, provenance, injection labeling	Exclude unsafe context or fail closed.
Instruction Guardrail	Versioned system/task instructions and policy separation	Use approved template only.
Generation Guardrail	Model route, limits, temperature/configuration, tool permissions	Block unapproved generation path.
Output Guardrail	Schema, semantic, policy, safety, identity validation	Reject invalid/unsafe output.
Tool Guardrail	Independent authorization and risk checks	Deny tool execution.
Candidate Guardrail	Target scope, version, policy, impact requirements	Do not create/advance unsafe candidate.
DAIL Guardrail	Verification, approval, promotion, trusted state	Block lifecycle transition.
Runtime Guardrail	Rate, quota, anomaly, circuit breaker, kill switch	Throttle, disable, or degrade.
Evidence Guardrail	Record safety decisions and material failures	Preserve forensic/audit trail.

4. Threat Model

Threat	Potential Impact	Required Guardrail
Direct prompt injection	Model ignores task/policy and produces unsafe output	Instruction hierarchy + output validation.
Indirect prompt injection	Cloud text/logs/documents manipulate model behavior	Untrusted-content isolation + deterministic checks.
Data exfiltration	Tenant or credential data leaves boundary	Minimization, redaction, output filtering.
Hallucination	Invented resource/change causes bad recommendation	Canonical identity validation + evidence requirements.
Tool abuse	Model invokes dangerous operation	Independent tool broker + least privilege.
Goal hijacking	User/content redirects task from authorized objective	Task contract + immutable policy context.

Threat	Potential Impact	Required Guardrail
Context poisoning	False facts cause incorrect reasoning	Provenance, freshness, deterministic source validation.
Model manipulation	Compromised/misconfigured model emits unsafe results	Provider/model allowlist + post-validation.
Jailbreak	Safety instructions bypassed	Defense in depth; model output never authoritative.
Denial of service	Expensive AI calls exhaust resources/budget	Quotas, rate limits, budgets, circuit breakers.
Cross-tenant inference	Model receives unauthorized tenant data	Pre-invocation authorization filtering.
Unsafe automation	Recommendation becomes infrastructure mutation	Candidate + DAIL + verification + governance.

5. Input Guardrails

5.1 User input

- Validate input length, encoding, content type, and request structure.
- Associate every AI request with an authorized task and tenant/scope.
- Treat user text as task input, not as an authority-bearing instruction.
- Prevent user input from modifying system-level policy, tool permissions, or model route.
- Rate-limit repeated or expensive requests.
- Reject requests attempting to retrieve credentials, secrets, or data outside authorization.

5.2 External content

- Cloud resource names, tags, descriptions, logs, documents, tickets, and integration payloads are untrusted content.
- External content must be clearly represented as data when passed to the model.
- Retrieved content cannot redefine system instructions or tool permissions.
- External content should carry source/provenance metadata.
- Content from untrusted sources should be sanitized or structurally encoded where practical.

6. Context Guardrails

Control	Requirement
Authorization filter	Only data within caller/tenant/scope authorization can enter context.
Data minimization	Send only fields necessary for the task.
Secret exclusion	Credentials, tokens, private keys, and sensitive secrets are excluded.
Provenance	Material facts identify source object/version where possible.
Freshness	Stale facts are labeled and safety-critical stale context is rejected.
Instruction separation	Policy/instructions are structurally separated from retrieved content.
Context budget	Deterministic size/relevance limits prevent uncontrolled context growth.
Context version	Material tasks retain a version/fingerprint of context

Control	Requirement
	recipe and source versions.

7. Prompt Injection Defense

7.1 Direct injection

Direct prompt injection occurs when a user attempts to override the task, safety policy, authorization, or intended objective through model-visible text. The system must assume the model may follow such text and therefore cannot rely on prompt wording alone.

- Keep system/task instructions outside user-controlled content.
- Use explicit task contracts and fixed output schemas.
- Do not expose internal policy secrets or hidden control information unnecessarily.
- Validate outputs independently of model instructions.
- Treat refusal or compliance behavior as an AI output, not a security decision.

7.2 Indirect injection

Indirect prompt injection can arrive through cloud resource descriptions, logs, documentation, repository content, tickets, or other retrieved sources.

- Label retrieved content as untrusted data.
- Never interpolate untrusted content into executable tool definitions or authorization rules.
- Require deterministic validation of any action suggested from retrieved content.
- Limit the model's ability to turn retrieved text into tool calls.
- Include malicious-content fixtures in regression evaluations.

8. Data Exfiltration Guardrails

- Do not provide credentials, secrets, session tokens, private keys, or unrestricted tenant datasets to the model.
- Filter output for accidental disclosure of secret-like values.
- Apply authorization to every retrieval before it enters model context.
- Prevent model-generated links or references from exposing unauthorized object identifiers.
- Restrict exports and reports to the user's authorized tenant/scope.
- Avoid retaining sensitive prompts/responses unnecessarily.
- Treat model output as potentially containing sensitive data and apply normal access controls.

9. Hallucination and Grounding Controls

9.1 Grounding requirements

- Claims about cloud resources should reference canonical asset identities.
- Security findings should reference deterministic rule/evidence sources.
- Recommendations should identify assumptions and missing facts.
- The model must distinguish observed facts from inference.
- Unknown information must remain unknown rather than being invented.
- High-impact recommendations require stronger evidence thresholds.

Scenario	Required Response
Unknown resource ID	Reject proposed change.
Resource exists but outside scope	Reject and record authorization failure.
Missing configuration field	Mark unknown; request/retrieve data or defer.
Conflicting evidence	Expose conflict; do not infer safe state.
Stale evidence	Require refresh for safety-critical decisions.
Unsupported provider feature	Do not invent syntax; return unsupported state.

10. Output Guardrails

10.1 Validation pipeline

1. Parse provider response.
2. Validate schema and required fields.
3. Reject unexpected or prohibited fields.
4. Validate canonical IDs and tenant/scope.
5. Validate referenced policy/rule/evidence identifiers.
6. Validate semantic constraints and permitted operation types.
7. Check for missing evidence and uncertainty.
8. Apply task-specific safety policy.
9. Normalize accepted values.
10. Only then hand output to the appropriate product workflow.

10.2 Output classes

Class	Examples	Control
Informational	Summary, explanation	Normal validation + access control.
Analytical	Risk rationale, dependency hypothesis	Evidence/provenance + uncertainty.
Recommendation	Suggested remediation	Schema + domain + policy validation.
Change proposal	Infrastructure diff/operation	Strict schema + identity + policy + impact + DAIL.
Tool request	Read/query/action request	Independent tool authorization.
Governance statement	Approval/safety assertion	Model cannot authoritatively create it.

11. Tool Safety

11.1 Tool risk classes

Risk	Example	Default Rule
T0 — None	Pure formatting/calculation	May execute within application.
T1 — Read-only	Read authorized cloud metadata	Broker + authorization.
T2 — Analytical	Run deterministic analysis/query	Broker + bounded execution.
T3 — Candidate-generating	Generate a proposed change artifact	Must create candidate; no execution.
T4 — Mutating	Change cloud resource	Excluded from direct model authority; governed DAIL path required.

11.2 Tool broker requirements

- Tool names and arguments are validated against a registry.
- Authorization is evaluated independently from model output.
- Tool scope is derived from trusted caller context.
- Tool execution uses least-privilege service identities.
- High-risk tools require explicit governance.
- Tool results are sanitized before returning to the model.
- Every request and outcome is auditable.
- Timeout, concurrency, and resource limits are enforced.

12. Remediation Safety

AI-assisted remediation is the highest-risk product intelligence path because its output can eventually affect real infrastructure. The safe architecture inserts explicit candidate, impact, verification, and governance boundaries.

11. AI produces a structured recommendation or proposed change.
12. CloudShield validates the target resource identities and scope.
13. A RemediationCandidate is created with immutable versioning.
14. Dependency and impact analysis runs against the exact candidate version.
15. Required verification checks execute.
16. Policy and human approval requirements are evaluated.
17. DAIL performs promotion gates.
18. Only governed promotion can invoke cloud mutation.
19. Successful promotion produces trusted state and evidence.

At no point does the model receive unrestricted production credentials or become the final authorization authority.

13. DAIL Safety Boundary

DAIL Control	AI Safety Requirement
Candidate identity	AI output cannot choose arbitrary identity; canonical resolution required.
Version binding	AI response is bound to exact candidate/context version.
Impact analysis	Model claims cannot replace deterministic impact analysis.
Verification	AI cannot mark a candidate verified.
Approval	AI cannot impersonate human/policy approval.
Promotion	AI cannot call promotion outside governed workflow.
Trusted state	AI cannot write trusted state.
Evidence	Material AI reasoning is recorded as supporting evidence, not proof.

DAIL is the deterministic safety boundary that converts an AI-assisted proposal into a governed infrastructure lifecycle. AI can assist every stage where useful, but cannot own the state transition.

14. Human-in-the-Loop Governance

Action	Human Requirement
Low-risk explanation	Not required.
Low-risk recommendation	Configurable; deterministic validation required.
Candidate creation	May be automated after validation, depending on policy.
High-impact candidate	Human review required.
Production promotion	Human/policy approval according to environment policy.
Emergency bypass	Privileged human action, explicit reason, expiry, and full audit.
Model/prompt safety-policy change	Engineering/security review required.

- The UI must clearly distinguish AI recommendation from approved action.
- Approval must identify the actor and exact candidate version.
- Approval becomes invalid when protected candidate/context changes.
- The system should present evidence, impact, verification status, and uncertainty before high-impact approval.

15. Policy Guardrails

Safety policies must be deterministic and versioned. Examples include allowed operation types, protected resources, environment restrictions, required approvals, maximum blast radius, credential rules, and mandatory verification checks.

Policy Type	Example Rule
Environment	Production mutation requires elevated approval.
Resource	Protected IAM/billing/security controls require stronger verification.
Operation	Destructive operations require explicit confirmation.
Blast radius	Changes affecting more than configured threshold require escalation.
Identity	Only authorized tenant/account/region targets permitted.
Evidence	Promotion blocked when required evidence is missing.
AI	Certain data classes may not be sent to external models.

16. Runtime Guardrails

- Per-user, per-tenant, per-task, and global rate limits.
- Maximum prompt/context and output sizes.
- Maximum AI job duration.
- Concurrency limits for expensive tasks.
- Provider/model allowlists.
- Circuit breakers for provider or task failures.
- Safety kill switch for a task, model route, tool, or AI capability.
- Automatic degradation to deterministic functionality where possible.
- Anomaly detection for unusual tool requests, output rejection rates, or cost spikes.

17. Safety Kill Switch

CloudShield AI shall provide an operational mechanism to disable an AI task, model route, tool, or the AI subsystem without disabling unrelated deterministic security functionality.

Scope	Effect
Task kill switch	Reject new executions for one AI task.
Model-route kill switch	Prevent use of a specific provider/model route.
Tool kill switch	Disable a tool immediately.
Tenant AI disable	Disable AI processing for one tenant where policy requires.
Global AI disable	Stop AI execution while discovery/security deterministic services remain available.

Disabling AI must not bypass DAIL or cause unsafe queued work to execute automatically.

18. Safety Event Model

Event	Severity Example	Response
Prompt injection detected	Medium/High	Reject/contain request; record event.
Unauthorized context access	High	Block request; security alert.
Secret-like output detected	High	Redact/block; investigate source.
Tool authorization denied	Medium/High	Block execution; audit.
Unsafe candidate rejected	High	Prevent lifecycle progression; preserve evidence.
DAIL gate bypass attempt	Critical	Block operation; alert and investigate.
Model route misconfiguration	High	Disable route; restore approved configuration.
AI cost anomaly	Medium	Throttle/disable affected route or tenant.

19. Safety Evidence and Audit

- Record safety policy version, guardrail decision, task ID, model route, correlation ID, tenant/scope, and affected object/version for material events.
- Do not log raw secrets or unnecessary sensitive prompts.
- Preserve evidence of rejected unsafe outputs when needed for incident analysis, subject to retention/privacy controls.
- Link safety events to candidate, verification, approval, and promotion identifiers when applicable.
- Make safety decisions queryable for operational review and evaluation.

20. Safety Evaluation Framework

20.1 Evaluation categories

Category	Examples
Injection resistance	Direct/indirect prompt injection, instruction smuggling.
Grounding	Invented resources, unsupported claims, missing

Category	Examples
	evidence.
Data protection	Secret extraction, cross-tenant requests, unauthorized retrieval.
Tool safety	Unauthorized tool calls, dangerous arguments, scope escalation.
Remediation	Destructive changes, malformed changes, excessive blast radius.
Policy adherence	Attempts to override environment/resource policies.
Robustness	Conflicting evidence, stale data, provider failures.
Consistency	Same scenario across model/prompt versions.

20.2 Release thresholds

- No critical safety bypass may be accepted for production.
- No known path may convert raw model output directly to trusted state.
- Critical tenant-isolation and secret-exfiltration tests must pass.
- High-risk remediation scenarios must pass deterministic validation and DAIL tests.
- Material regression in safety metrics blocks automatic model/prompt rollout.
- Exceptions require explicit risk acceptance and expiration.

21. Red-Team Test Suite

- Ignore-previous-instructions attacks.
- Malicious cloud resource descriptions instructing the model to change security settings.
- Logs containing fake tool commands.
- User attempts to retrieve another tenant's infrastructure.
- User attempts to reveal provider/API credentials.
- Model-generated change targeting nonexistent resource IDs.
- Model-generated destructive operation hidden in benign description.
- Candidate mutation after verification.
- Dependency graph poisoning intended to hide blast radius.
- Tool-call argument manipulation.
- Replay of previously approved candidate.
- Model output containing unsupported policy claims.
- Provider response containing unexpected fields or malicious content.

22. Safety in Multi-Model Systems

If multiple models are later introduced, each model remains untrusted. A reviewer model cannot become a substitute for deterministic validation simply because two models agree.

- Model agreement is evidence of consistency, not proof of correctness.
- Different models must receive only authorized context.
- Reviewer outputs remain advisory unless deterministic controls accept the underlying object.
- Model ensembles must preserve task, schema, policy, and DAIL boundaries.
- Model diversity should be used for evaluation/review, not authority escalation.

23. Safety in Local Models

- Local deployment reduces some external-provider exposure but does not remove prompt injection, hallucination, tool abuse, or privilege risks.
- The same output validation and DAIL controls apply.
- Model artifacts and runtime images become part of the supply-chain security boundary.
- Local models must be versioned, evaluated, and allowlisted before production use.
- A local model must not gain broader cloud authority than a hosted model.

24. Safe Degradation

Condition	Safe Degraded Behavior
LLM unavailable	Continue deterministic inventory/security analysis.
Guardrail service unavailable	Block affected AI/safety-critical workflow.
Context source unavailable	Return partial deterministic state with explicit unknowns or block.
Model quality regression	Route to approved fallback or disable affected task.
Tool broker unavailable	Deny tool execution.
Policy service unavailable	Block mandatory governed actions.
Evidence recorder unavailable	Block or enter explicit recovery state for security-critical transitions.

25. Configuration and Versioning

Artifact	Versioning Requirement
Safety policy	Versioned and audited.
Guardrail configuration	Versioned; controlled deployment.
Prompt template	Versioned and evaluated.
Output schema	Versioned with compatibility policy.
Model route	Versioned configuration.
Tool registry	Versioned; each tool has risk class.
Evaluation suite	Versioned dataset and expected outcomes.
Safety exception	Explicit owner, reason, scope, expiry.

26. Incident Response for AI Safety Events

20. Detect and classify the safety event.
21. Disable affected task/model/tool route if active exploitation is suspected.
22. Identify tenant, request, model, prompt/configuration, and affected object versions.
23. Contain identities or integrations involved.
24. Preserve relevant evidence.
25. Determine whether any candidate, verification, promotion, or cloud state was affected.
26. Reconcile external cloud state if execution is ambiguous.
27. Correct the guardrail/prompt/model/configuration defect.
28. Run regression and red-team tests.
29. Re-enable only after safety criteria pass and the incident is documented.

27. MVP Safety Baseline

- All AI output treated as untrusted until application validation.
- Provider-neutral LLM integration with approved model routes.
- Tenant-aware context filtering before provider calls.
- No credentials or unrestricted secrets in model context.
- Direct and indirect prompt-injection defenses.
- Strict structured output validation.
- Canonical asset identity validation for infrastructure recommendations.
- No direct model-to-cloud mutation authority.
- Read-only tool capability only if required; mutation excluded from MVP.
- Explicit RemediationCandidate creation before DAIL.
- Impact, verification, approval, and promotion gates remain deterministic.
- AI safety event telemetry and kill switch.
- Golden/adversarial/red-team evaluation suite.
- Safe degraded behavior during provider failure.

28. Future Guardrail Capabilities

- Dedicated AI policy decision service.
- Advanced content provenance and taint tracking.
- Model-specific adversarial classifiers.
- Continuous runtime prompt-injection detection.
- Human approval workflows with richer risk-adaptive thresholds.
- Independent reviewer models for high-risk recommendations.
- Formal policy-as-code integration.
- Cryptographic attestation of model/configuration artifacts.
- Automated continuous red teaming.
- Private/local model isolation and confidential inference where justified.

29. Security Acceptance Criteria

30. No prompt or retrieved content can alter server-side authorization.
31. No raw cloud credential can enter model context through a tested product path.
32. Cross-tenant data cannot be included in an AI request outside authorization.
33. Prompt injection and indirect prompt injection test suites are part of release gates.
34. A model cannot directly mutate production infrastructure.
35. Tool calls are independently authorized and scoped.
36. Malformed or semantically invalid AI output cannot create a valid infrastructure candidate.
37. AI cannot mark a candidate verified, approved, promoted, or trusted.
38. Safety-critical workflows fail closed when required guardrail/policy controls are unavailable.
39. Safety events are attributable, observable, and linked to relevant lifecycle objects.

30. Functional Acceptance Criteria

40. The system rejects an AI response containing an unknown resource identifier.
41. The system rejects a recommendation outside the caller's tenant/account/region scope.
42. The system identifies missing information rather than accepting fabricated values.

43. The system prevents a retrieved malicious instruction from changing tool permissions.
44. The system can disable an AI task without disabling deterministic security analysis.
45. The system can route a validated infrastructure proposal into DAIL without granting the model promotion authority.
46. The system invalidates affected safety decisions when protected candidate/context versions change.
47. The system records safety guardrail outcomes for material AI interactions.
48. The system can operate in a deterministic degraded mode when the LLM provider is unavailable.

31. Traceability

Source	Safety Dependency
00 — Master System Specification	AI role, governance, trusted-state model, DAIL positioning.
01 — Software Requirements Specification	AI abstraction, structured output, security, validation, human governance.
02 — System Architecture Specification	Service boundaries, data flows, trust boundaries, async execution.
03 — Functional Specification	AI, remediation, candidate, verification, approval, promotion behavior.
04 — Product & UX Specification	Safe-path UX, uncertainty, confirmations, stale/error states.
05 — Domain & Data Model	AI requests/responses, candidate, evidence, policy, lifecycle entities.
06 — State & Lifecycle Specification	Version binding, invalidation, fail-closed transitions, reconciliation.
07 — Identity & Dependency Specification	Stable IDs, dependency provenance, impact and unknown semantics.
08 — Security & Threat Model	AI threats, tenant isolation, secrets, IAM, evidence, supply chain.
09 — Intelligence & AI Architecture	Context, tasks, validation, tool boundary, model abstraction.
10 — LLM Integration Specification	Provider adapters, requests/responses, retries, routing, credentials, observability.

32. Engineering Decisions

Decision	Rationale
AI is untrusted	Prevents model confidence from becoming infrastructure authority.
Deterministic validation after generation	Separates probabilistic reasoning from correctness/safety enforcement.
No direct mutation tools in MVP	Minimizes blast radius and implementation complexity.
DAIL remains final lifecycle safety boundary	Ensures verification and promotion are deterministic/governed.
Context is authorized before generation	Prevents AI from becoming an alternate data-access path.
Kill switch required	Allows rapid containment without disabling deterministic product capabilities.
Safety evaluation is a release gate	Prevents model/prompt changes from silently weakening controls.
Local models inherit same guardrails	Changing deployment topology must not change

Decision	Rationale
	security authority.

33. Definition of Done

- Input, context, output, tool, candidate, and runtime guardrails are implemented or explicitly scoped for MVP.
- Prompt injection and indirect injection test cases exist.
- Secret and cross-tenant leakage tests pass.
- Structured output validation is implemented outside the model.
- Canonical identity and scope checks protect candidate generation.
- No model path can directly invoke unrestricted cloud mutation.
- DAIL integration preserves exact candidate/version and deterministic gates.
- Safety events and guardrail decisions are observable.
- AI task/model/tool kill switch is operational.
- Golden, adversarial, and red-team evaluation suites are versioned.
- Provider outage produces safe degraded behavior.
- Safety configuration and exceptions are versioned and auditable.

34. Glossary

Term	Definition
AI Guardrail	Control that constrains, validates, monitors, or blocks AI behavior.
Prompt Injection	Attempt to manipulate model behavior through user or model-visible instructions.
Indirect Prompt Injection	Malicious instructions embedded in retrieved external content.
Grounding	Constraining AI claims to authorized, relevant, traceable facts.
Tool Broker	Independent authorization boundary for model-requested tool execution.
Safety Profile	Named set of task-specific AI safety controls.
Kill Switch	Operational control that disables an AI task, route, tool, tenant capability, or the AI subsystem.
RemediationCandidate	Explicit infrastructure change proposal subject to deterministic validation and DAIL lifecycle.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem.
Trusted State	State earned only after governed successful promotion.
Safe Degradation	Behavior that preserves deterministic product functionality while disabling or limiting unsafe AI capabilities.